



Advanced Web Programming

Ung Văn Giàu
Email: giau.ung@eiu.edu.vn



Reusable Components

Topic

We'll cover the following topics:

- Creating the project
- Using generic props
- Using props spreading
- Using render props
- Adding checked functionality



Technical requirements

- Node.js and npm
- Visual Studio Code



Creating the project

- Create a project
- Install Tailwind CSS into the project and configure it



Using generic props

Understanding generics

- For example, the **useState** hook has an optional generic parameter for the type of state variable:

```
const [visible, setVisible] = useState<boolean>()
```

- Generic parameters in a function allow that function to be reusable with different types and be strongly typed.
- The following function only works with a string array:

```
function first(array: Array<string>): string | null {  
    return array.length === 0 ? null : array[0];  
}
```

Using generic props

Understanding generics

Generic functions

- Generic type parameters are defined in angled brackets before the function's parentheses:

```
function someFunc<T1, T2, ...>(...) {  
    ...  
}
```

- Here is a generic version of the function we saw earlier.

```
function first<Item>(array: Array<Item>): Item | null {  
    return array.length === 0 ? null : array[0];  
}
```

Using generic props

Understanding generics

Generic types

- Custom types can be generic as well. For a **type** alias, its generic parameters are defined in angled brackets after the type name:

```
type TypeName<T1, T2, ...> = {  
  ...  
}
```

- For example, the props of a React component can be generic.

```
type Props<Item> = {  
  items: Item[];  
  ...  
};
```

Using generic props

Understanding generics

Generic React components

- Generic props can be integrated into a generic function to produce a generic React component.

```
type Props<Item> = {  
  items: Item[];  
};  
  
export function List<Item>({ items }: Props<Item>) {  
  ...  
}
```

Creating a basic list component

- Carry out the following steps:
 - Create a folder for the component called **Checklist** in the **src** folder. Then, create a file called **Checklist.tsx** in this folder.
 - Open **Checklist.tsx** and add the following **Props** type:

```
type Props<Data> = {  
  data: Data[];  
  id: keyof Data;  
  primary: keyof Data;  
  secondary: keyof Data;  
};
```

- The **keyof** operator queries the type specified after it for the property names and constructs a union type from them

Creating a basic list component

- Carry out the following steps:
 - Implement the component function

```
export function Checklist<Data>({
  data,
  id,
  primary,
  secondary,
}: Props<Data>) {
  return (
    <ul className="bg-gray-300 rounded p-10">
      {data.map((item) => {
      })}
    </ul>
  );
}
```

Creating a basic list component

- Carry out the following steps:
 - Implement the function inside **data.map**

```
{data.map((item) => {  
  const idValue = item[id] as unknown;  
  if (  
    typeof idValue !== 'string' &&  
    typeof idValue !== 'number'  
  ) {  
    return null;  
  }  
  const primaryText = item[primary] as unknown;  
  if (typeof primaryText !== 'string') {  
    return null;  
  }  
  const secondaryText = item[secondary] as unknown;  
}
```

Creating a basic list component

- Carry out the following steps:
 - Finish the implementation by rendering the list item.
 - Create a new file in the **Checklist** folder called **index.ts** and export the **Checklist** component into it:

`export * from './Checklist';`

```
{data.map((item) => {  
  ...  
  return (  
    <li  
      key={idValue}  
      className="bg-white p-6 shadow rounded mb-4  
        last:mb-0"  
    >  
      <div className="text-xl text-gray-800 pb-1">  
        {primaryText}  
      </div>  
      {typeof secondaryText === 'string' && (  
        <div className="text-sm text-gray-500">  
          {secondaryText}  
        </div>  
      )}  
    </li>  
  );  
})}
```

Creating a basic list component

- Carry out the following steps:
 - Open **App.tsx** and replace the content within it with the following.
 - Run the app.

```
import { Checklist } from './Checklist';

function App() {
  return (
    <div className="p-10">
      <Checklist
        data={[
          { id: 1, name: 'Lucy', role: 'Manager' },
          { id: 2, name: 'Bob', role: 'Developer' },
        ]}
        id="id"
        primary="name"
        secondary="role"
      />
    </div>
  );
}

export default App;
```

Creating a basic list component

Recap

- TypeScript generics allow reusable code to be strongly typed.
- Functions can have generic parameters that are referenced within the implementation.
- Types can also have generic parameters that are referenced within the implementation.
- A React component can be made generic by feeding a generic props type into a generic function component. The component implementation will then be based on generic props.

Using props spreading

- **props spreading** pattern is useful when you want to use all the props from an HTML element in the implementation of a component.
- Carry out the following steps:
 - Open **Checklist.tsx** and import the following type from React:

```
import { ComponentPropsWithoutRef } from 'react';
```

This type allows us to reference the props of an HTML element such as `ul`. It is a generic type that takes the HTML element name as a generic parameter.

Using props spreading

- Carry out the following steps:
 - Add the props from the **ul** element to the component props type

```
type Props<Data> = {  
  data: Data[];  
  id: keyof Data;  
  primary: keyof Data;  
  secondary: keyof Data;  
} & ComponentPropsWithoutRef<'ul'>;
```

Using props spreading

- Carry out the following steps:
 - Add a **rest parameter** called **ulProps** to collect all the props for the **ul** element into a single ulProps variable:

```
export function Checklist<Data>({  
  data,  
  id,  
  primary,  
  secondary,  
  ...ulProps  
}: Props<Data>) {  
  ...  
}
```

Using props spreading

- Carry out the following steps:
 - Spread **ulProps** onto the **ul** element using the spread syntax:

```
export function Checklist<Data>({
  data,
  id,
  primary,
  secondary,
  ...ulProps
}: Props<Data>) {
  return (
    <ul
      className="bg-gray-300 rounded p-10"
      {...ulProps}
    >...</ul>
  );
}
```

Using props spreading

- Carry out the following steps:
 - We can use this new feature of **Checklist** to specify the list height and width. Open **App.tsx** and add the following style attribute, as well as more data items.
 - Run the app.

```
<Checklist
  data={ [
    { id: 1, name: 'Lucy', role: 'Manager' },
    { id: 2, name: 'Bob', role: 'Developer' },
    { id: 3, name: 'Bill', role: 'Developer' },
    { id: 4, name: 'Tara', role: 'Developer' },
    { id: 5, name: 'Sara', role: 'UX' },
    { id: 6, name: 'Derik', role: 'QA' }
  ] }
  id="id"
  primary="name"
  secondary="role"
  style={{
    width: '300px',
    maxHeight: '380px',
    overflowY: 'auto'
  }}
/>
```

Using props spreading

- We intersect the props type with **ComponentPropsWithoutRef** to add props for the HTML element we want to spread onto
- We use a rest parameter in the component props to collect all the HTML element props into an array
- We can then spread the rest parameter onto the HTML element in the JSX

Using render props

Understanding the render props pattern

- A way of making a component highly reusable is to allow the consumer to render internal elements within it.
- The children prop on a **button** element is an example of this because it allows us to specify any button content we like:

```
<button>We can specify any content here</button>
```

- The render props pattern allows us to use a prop other than **children** to provide this capability.

```
<Modal heading={<h3>Enter Details</h3>}>  
  Some content  
</Modal>
```

Using render props

Understanding the render props pattern

- Carry out the following steps:
 - Render props are useful when allowing the consumer to render elements associated with the data passed into the component because the render prop can be a function:

```
<List  
  data={ [...]}  
  renderItem={ (item) => <div>{item.text}</div> }  
</>
```

Using render props

Adding a renderItem prop

- Carry out the following steps:
 - Open **Checklist.tsx** and add the **ReactNode** type to the React **import** statement:

```
import { ComponentPropsWithoutRef, ReactNode } from  
'react';
```

ReactNode represents an element that React can render.

Using render props

Adding a renderItem prop

- Carry out the following steps:
 - Add a render prop called **renderItem** to the **Props** type:

```
type Props<Data> = {  
  data: Data[];  
  id: keyof Data;  
  primary: keyof Data;  
  secondary: keyof Data;  
  renderItem?: (item: Data) => ReactNode;  
} & React.ComponentPropsWithoutRef<'ul'>;
```

Using render props

Adding a renderItem prop

- Carry out the following steps:
 - Add **renderItem** to the component function parameters:

```
export function Checklist<Data>({
  data,
  id,
  primary,
  secondary,
  renderItem,
  ...ulProps
}: Props<Data>) {
  ...
}
```

Using render props

Adding a renderItem prop

- Carry out the following steps:
 - In the JSX, at the top of the mapping function, add an if statement to check whether the renderItem prop has been specified. If renderItem has been specified, call it with the data item, and return its result from the mapping function:

```
<ul ...>
  {data.map((item) => {
    if (renderItem) {
      return renderItem(item);
    }
    const idValue = item[id] as unknown;
    ...
  })}
</ul>
```

Using render props

Adding a renderItem prop

- Carry out the following steps:
 - To try the new prop out, open **App.tsx** and add the following renderItem attribute.
 - Run the app.

```
<Checklist
  ...
  renderItem={(item) => (
    <li key={item.id} className="bg-white p-4
      border-b-2">
      <div className="text-xl text-slate-800 pb-1">
        {item.name}
      </div>
      <div className="text-slate-500">{item.role}</div>
    </li>
  )}
/>
```

Using render props

Recap

- The render props pattern allows a component consumer to override the rendering of parts of the component
- A render prop can either be an element or a function that returns an element
- A common use case for a render prop is a data-driven list in which the rendering of list items can be overridden

Adding checked functionality

- Carry out the following steps:
 - Open **Checklist.tsx** and add **useState** to the React **import** statement:

```
import {  
  ComponentPropsWithoutRef,  
  ReactNode,  
  useState  
} from 'react';
```

Adding checked functionality

- Carry out the following steps:
 - At the top of the component implementation, add the state for the IDs of the checked items:

```
const [checkedIds, setCheckedIds] = useState<IdValue[]>([]);
```

Adding checked functionality

- Carry out the following steps:
 - Add checkboxes to the list of items

```
<li
  key={idValue}
  className="bg-white p-6 shadow rounded mb-4 last:mb-0"
>
  <label className="flex items-center">
    <input
      type="checkbox"
      checked={checkedIds.includes(idValue)}
      onChange={handleCheckChange(idValue)}
    />
    <div className="ml-2">
      <div className="text-xl text-gray-800 pb-1">
        {primaryText}
      </div>
      {typeof secondaryText === 'string' && (
        <div className="text-sm text-gray-500">
          {secondaryText}
        </div>
      )}
    </div>
  </label>
</li>
```

Adding checked functionality

- Carry out the following steps:
 - Implement the **handleCheckChange** function in the component

```
const [checkedIds, setCheckedIds] =  
  useState<IdValue []> ( [] );  
  
const handleCheckChange = (checkedId: IdValue) => () =>  
  {};  
  
return ...
```

This is a function that returns the handler function. This complexity is because a basic checked handler doesn't pass in the list item's ID. This approach is called **currying**.

Adding checked functionality

- Carry out the following steps:
 - Complete the handler implementation

```
const handleCheckChange = (checkedId: IdValue) => () => {  
  const isChecked = checkedIds.includes(checkedId);  
  let newCheckedIds = isChecked  
    ? checkedIds.filter(  
      (itemCheckedid) => itemCheckedid !== checkedId  
    )  
    : checkedIds.concat(checkedId);  
  setCheckedIds(newCheckedIds);  
};
```

Adding checked functionality

- Carry out the following steps:
 - Create a new file in the **Checklist** folder called **types.ts** with the definition of **IdValue**:

```
export type IdValue = string | number;
```
 - Move back to **Checklist.tsx** and import **IdValue**:

```
import { IdValue } from './types';
```
 - Run the app.



Q&A